

UART Single-Wire LED Display Interface Driver Library

Technical Specification

Version 4.x

Overview

The BC759x series LED display driver + keyboard interface chips provide a unified UART single-wire LED display interface. This driver library is applicable to all chips in this series.

The driver library supports both interrupt-driven and non-interrupt UART operation modes. Non-interrupt mode applies when the MCU does not have a hardware UART and uses software-simulated UART, or when the MCU has a hardware UART but its interrupt function is not used. When the UART interface uses interrupt-driven operation, the driver library provides an internal FIFO buffer with a user-definable size. UART transmission is then completed in the background under the control of the driver library. This minimizes the processor time occupied by display operations and saves more resources for higher-priority user tasks.

The driver library is written in standard C and is suitable for all target environments that use the C language. It is also lightweight. Taking a Cortex-M3 target environment as an example, after compilation this library occupies only about 560 bytes of program space and 36 bytes of RAM when interrupt mode is used with an 8-byte FIFO buffer.

Each BC759x command consists of two bytes: the first byte is the command, and the second byte is the data. The driver library provides a basic operation function, `send_cmd()`, which can be used to send any command to the BC759x. The library also provides several higher-level functions that wrap the most commonly used display operations:

- `clear()` - Clear the display and blinking status
- `display_dec()` - Display a value in decimal format
- `display_hex()` - Display a value in hexadecimal format
- `display_float()` - Display a value in float format
- `digit_blink()` - Control blinking display by digit position

Some functions, although commonly used - such as turning a single LED on or off - can be completed with a single BC759x command. Therefore, no additional higher-level wrapper

functions are provided for these functions, because doing so would not provide much benefit for the user.

Installing the Library

Installing the library is very simple. The simplest method is to copy all header files (`*.h`) and the source file `led_disp.c` from the driver library into the project directory, placing them together with the main program file. Then add `led_disp.c` to the project build list and include the `led_disp.h` header file in the user source file.

After including the `led_disp.h` header file in the user's C source file, the functions in the driver library can be used directly in the user program.

Configuring the Driver Library

All user-controllable configuration options of the driver library are located in the `disp_config.h` file. There are three configurable options:

LOW_DIG_NUM_ON_RIGHT

This option affects the display direction of the decimal and hexadecimal display functions.

Because of differences in PCB layout, the arrangement of the 7-segment display digits on the board may vary. If lower-numbered digits such as `DIG0` and `DIG1` are located on the right side of a row of display digits, set this option to `1` (default). If lower-numbered digits are located on the left side, this option should be set to `0`.

UART_MODE_INTERRUPT

Set this option according to whether the UART is used in interrupt mode:

- `1` : Use interrupt mode. This is the default value.
- `0` : Do not use interrupt mode.

When interrupt mode is used, the driver library provides a built-in UART FIFO buffer, allowing UART data transmission to run in the background and improving program efficiency.

UART_FIFO_SIZE

This option is meaningful only in interrupt mode. It defines the size of the UART FIFO buffer. The value must be a power of 2, such as 2, 4, 8, 16, 32, and so on.

Function Reference

`set_write_func()` - Set the UART Write Function

Function declaration:

```
void set_write_func(void (*pUartWriteFunc)(uint8_t));
```

The user must wrap the UART write operation into a function. This function must take one `uint8_t` parameter as input; the input parameter is the data to be sent. The return value must be `void`. Before using the driver library, the user must call this function to tell the driver library which function is the UART write function.

When interrupt mode is used, the write function may return immediately after writing to the UART register. In non-interrupt mode, the write function must confirm that the UART register has been cleared before writing, or wait to return until data transmission is complete after writing. Otherwise, the UART may not work correctly.

`set_eni_func()` - Set the UART Interrupt Enable Function

Function declaration:

```
void set_eni_func(void (*pTxIntEnFunc)(void));
```

This function is required only when interrupt mode is used. When the driver library uses UART interrupts, it must be able to enable and disable UART interrupts. If the UART transmit and receive functions of the chip use different interrupts, this function may control either the overall UART interrupt or the UART transmit interrupt. The UART transmit interrupt should occur only when UART data transmission has completed.

The user must wrap the interrupt enable operation into a function and then call this library function to tell the driver library which function should be called to enable the UART interrupt.

The interrupt enable function must take `void` as input and return `void`.

set_disi_func() - Set the UART Interrupt Disable Function

Function declaration:

```
void set_disi_func(void (*pTxIntDisFunc)(void));
```

This function is required only when interrupt mode is used. When the driver library uses UART interrupts, it must be able to enable and disable UART interrupts. If the UART transmit and receive functions of the chip use different interrupts, this function may control either the overall UART interrupt or the UART transmit interrupt. The UART transmit interrupt should occur only when UART data transmission has completed.

The user must wrap the interrupt disable operation into a function and then call this library function to tell the driver library which function should be called to disable the UART interrupt.

The interrupt disable function must take `void` as input and return `void`.

tx_ready() - UART Transmission Ready

Function declaration:

```
void tx_ready(void);
```

This function is required only when interrupt mode is used. In interrupt mode, the UART generates an interrupt after data transmission is complete. This function should be called from that interrupt service routine to notify the driver library that it may write the next byte to the UART if the FIFO is not empty.

send_cmd() - Send a Command

Function declaration:

```
void send_cmd(uint8_t Cmd, uint8_t Data);
```

This function can be used to send any command to the BC759x. The library provides only a few commonly used higher-level display functions. If the user needs full control of the BC759x, this function should be called directly.

This function has two parameters:

- `Cmd` : the command to be sent
- `Data` : the data to be sent

The `led_disp.h` header file lists all command definitions for the BC759x chips. For detailed descriptions of the commands for a specific BC759x chip, refer to the technical manual of the selected BC759x chip.

All other functions provided by the library implement their functionality by calling this function.

`clear()` - Clear Display and Blinking

Function declaration:

```
void clear(void);
```

This function clears all display and blinking attributes.

`display_dec()` - Display a Decimal Value

Function declaration:

```
void display_dec(uint32_t Val, uint8_t Pos, uint8_t Width);
```

This function displays the input value in decimal format. It accepts only unsigned values. If a negative value needs to be displayed, the user must display the `-` sign separately. If higher-order digits exceed the chip's display range, the excess digits will not be displayed, and no error message will be reported.

`Val` is the value to be displayed, with a range of `0` to `4,294,967,295`.

`Pos` is the display position. The display position is based on the least significant digit. The value of `Pos` is the `DIG` number where the least significant digit is located, with a range of `0` to `31`.

`Width` is the display width. The lower 7 bits set the display width, with a range of `1` to `32`. The highest bit determines whether leading zeros are displayed.

When the display width is smaller than the actual width of the value, the excess higher-order digits are not displayed. When `width` is greater than the actual width of the value:

- If the highest bit of `width` is `0`, the extra positions are blank and display nothing.
- If the highest bit of `width` is `1`, the extra positions display `0`.

For example, if the actual value is `123` and the display width `width` is set to `5`, the display result is:

```
__123
```

where `_` represents a blank display position.

If `width` is set to `0x85`, the display result is:

```
00123
```

If higher-order digits exceed the actual display range of the chip, the excess portion is ignored.

The `LOW_DIG_NUM_ON_RIGHT` option in the `disp_config.h` configuration file affects the direction of numerical display. When this value is `1`, digit `Pos` displays the least significant digit, and the higher-order digits of the displayed value are sequentially displayed on `DIG` positions with higher index values. When this value is `0`, the higher-order digits of the displayed value are sequentially displayed on `DIG` positions with index values smaller than `Pos`. This setting can be used to adapt to different PCB designs.

`display_hex()` - Display a Hexadecimal Value

Function declaration:

```
void display_hex(uint16_t Val, uint8_t Pos, uint8_t Width);
```

This function displays the input value in hexadecimal format. The maximum input value for this function is `0xffff`. Larger numbers can still be displayed by splitting the value and calling this function multiple times.

The meanings of `Pos` and `Width` are the same as in `display_dec()`. The difference is that, for hexadecimal display, when `width` is set greater than the actual width of the value, the extra positions display `0` instead of the blank positions used in decimal display.

For example, if the input value is `0xA5` and `Width` is set to `5`, the display result is:

```
000A5
```

`display_float()` - Display a Floating-Point Value

Function declaration:

```
void display_float(float Val, uint8_t Pos, uint8_t Width, uint8_t Frac);
```

This function directly accepts a value of the `float` data type through `Val` and displays it. The least significant digit after the decimal point is automatically rounded.

The number of digits after the decimal point is determined by `Frac`. The valid range of `Frac` is `0` to `7`.

The position and width of the overall display area are determined by `Pos` and `Width`. Their meanings are the same as the corresponding parameters in `display_dec()`. `Width` must not be smaller than `Frac`.

This function does not display the decimal point or the minus sign. If needed, the user must use additional commands to display the decimal point and minus sign. In this way, when the decimal point position is fixed, the user can use a fixed circuit to keep the decimal point constantly on, while using the chip's decimal point output to drive other indicator LEDs.

The following examples illustrate the use of the control parameters. For clarity, the display results in these examples include the decimal point:

- Example 1: If `Val = 3.14`, `Width = 5`, and `Frac = 4`, the display result is `3.1400`.
- Example 2: If `Val = 3.14159`, `Width = 7`, and `Frac = 4`, the display result is `_3.1416`, where `_` represents a blank display position.
- Example 3: If `Val = 0.00567`, `Width = 0x88`, and `Frac = 3`, the display result is `00000.006`.
- Example 4: If `Val = 476.0056`, `Width = 4`, and `Frac = 3`, the display result is `6.006`, because the higher-order digits are not displayed due to exceeding the display width.

Internally, this function converts the floating-point number to be displayed into a 32-bit unsigned integer and then calls `display_dec()`. Therefore, care must be taken to prevent 32-bit integer overflow.

For example, if the floating-point value is `Val = 500000.3` and `Frac = 4` is used for display, the function must first convert the data internally to the unsigned integer `5000003000`. However, the maximum value representable by a 32-bit unsigned integer is `4,294,967,295`, so overflow will occur and the correct result cannot be obtained.

`digit_blink()` - Control Blinking by Digit Position

Function declaration:

```
void digit_blink(uint8_t Digit, uint8_t OnOff);
```

This function controls blinking by digit position. Each call controls the blinking attribute of one display digit.

For blinking control of digits 16 to 31, if the user directly changes the state by calling `send_cmd()`, then later uses this function to control blinking for digits 16 to 31, the settings made by the direct command may be lost.

`Digit` is the display digit to be configured, with a range of `0` to `31`.

`OnOff` is the state to be set:

- `1` : blinking enabled
- `0` : blinking disabled

Usage Examples

Without Interrupt Mode

This application performs counting and displays the counter value in decimal format. Refer to the following pseudo-code:


```

#include "led_disp/led_disp.h"

uint16_t Counter;

int main(void)
{
    set_write_func(write_uart);

    while (1)      /* Main loop */
    {
        do_something();    /* Main program task */
        ...
        disp_dec(Counter, 0, 5); /* Display the counter value in decimal format starting from
digit 0 with a width of 5 digits */
        Counter++;
        delay();
    }
}

void write_uart(uint8_t TxData)
{
    TX_REG = TxData;      /* Write data to the transmit register */
    while (!Tx_Ready);    /* Confirm that transmission has completed */
}

```

With Interrupt Mode

The task is the same as in the previous example. Refer to the following pseudo-code:

```

#include "led_disp/led_disp.h"

uint16_t Counter;

int main(void)
{
    set_write_func(write_uart);
    set_eni_func(enable_tx_interrupt);
    set_disi_func(disable_tx_interrupt);

    while (1)      /* Main loop */
    {
        do_something();    /* Main program task */
        ...
        disp_dec(Counter, 0, 5); /* Display the counter value in decimal format starting from
digit 0 with a width of 5 digits */
        Counter++;
        delay();
    }
}

void write_uart(uint8_t TxData)
{
    while (!Tx_Ready);    /* Confirm that the transmit register has been cleared */
    TX_REG = TxData;      /* Write data to the transmit register */
}

void enable_tx_interrupt(void)
{
    TXIE = 1;
}

void disable_tx_interrupt(void)
{
    TXIE = 0;
}

void UART_ISR(void)      /* UART interrupt service routine */
{
    if (TX)                /* If this is a transmit interrupt */
    {
        tx_ready();
    }
}

```

License

Please refer to the license file or distribution terms included with this repository.

Copyright © Beijing Bitcode Technology Co., Ltd.

Website: <https://bitcode.com.cn/en/>